אוניברסיטת חיפה
UNIVERSITY OF HAIFA
جامعة حيفا

BIG DATA Course

Homework 2

Doing Analytics with Spark

Lecturer:

Artem Barger

Submitted by:

- Yousef Shehade
- Shada Esawi

Submission date: 01.03.2026

Project Overview

This project leverages Apache Spark (via PySpark) to process, clean, and analyze the IMDB dataset. The objective is to extract meaningful insights regarding movie and television trends, genre diversity, and actor collaboration networks using distributed computing principles.

Environment Setup & Execution

1. Open the provided BigDataHW2.ipynb notebook in Google Colab.
2. Upload the IMBD.csv dataset to the active Colab session storage.
3. Run the first cell to install the PySpark dependencies (!pip install pyspark).
4. Execute the cells sequentially. Spark will build a DAG for the transformations and execute only when actions (e.g., `show()`, `count()`) are called. The session remains active until the notebook runtime ends.

```
from pyspark.sql import SparkSession

# Initialize Spark
spark = SparkSession.builder.appName("IMDB Big Data Analytics") \
    .master("local[*]").getOrCreate()

# Load the dataset
df = spark.read.csv("IMBD.csv", header=True, inferSchema=True, quote='"', escape='"')

# Peek at the initial data types before we start cleaning
print("Raw Data Schema:")
df.printSchema()
```

** Link to Colab file **:

https://colab.research.google.com/drive/1NBccqFDzYU7UpipddvB8r2MuRXYv51Yq?usp=sharing

Methodology, Insights, and Analysis

Task 1: Data Cleaning and Preprocessing

Handled missing values by dropping rows where critical identifiers ($title, rating, year, genre, votes$) were null. Transformed the votes and duration strings into Integer types by utilizing $regexp_replace$ to strip out commas and text characters. We extracted a clean year format ($year_str$) and derived a numeric $start_year$ using $regexp_extract(\backslash d\{4\})$. To accurately distinguish TV Shows from Movies, we created a boolean flag (is_tv_show) by detecting year ranges such as "2015–2020" or "2018–" using an rlike regular expression that supports hyphen, en dash, and em dash characters ([-–—]).

Optimization: Cached df_clean in memory. Spark evaluates lazily, so caching at this point prevents the DAG from re-processing these heavy string manipulations during every subsequent task query.

Insight: Cleaning and standardizing the dataset ensured reliable numerical analysis and prevented errors caused by inconsistent formatting, enabling accurate aggregation and analysis.

Task 2: Top Rated Movies by Genre

Filtered the cached dataset strictly for movies. We utilized PySpark's `explode` function to unnest the comma-separated genre strings so movies belonging to multiple genres were accurately represented in each respective category and applied `trim` to remove extra whitespace. A Window function partitioned the data by genre and ordered it by rating (descending). We applied secondary sorting by votes and title to ensure consistent "tie breaking", yielding the top 10 movies per genre.

Task 3: Actor Collaboration Network

The raw stars column contained structural artifacts (e.g., bracket lists, director names, and 'Stars:' prefixes). We applied $regexp_replace$ to clean these artifacts, then `split` and `exploded` the column to extract individual actor names. To avoid duplicate actor entries per title, we used `distinct`. We executed a self-join using both ($title, start_year$) to map actor collaborations within the same production. By enforcing $a1.actor < a2.actor$, we prevented duplicate mirrored pairs (Actor A & Actor B vs. Actor B & Actor A).

Task 4: High-Rated Hidden Gems

We defined a "hidden gem" algorithmically as any movie with a $rating > 8.0$ and total $votes < 10{,}000$. We `filter`ed the dataset to include only movies and applied these logical constraints to isolate highly rated titles with relatively low popularity. To avoid duplicate entries, we used `distinct()`. The results were ordered descending by rating and ascending by votes to prioritize the highest-rated films with the smallest audience footprint.

Insight: Many hidden gems belong to drama, adventure, and action genres and have very high ratings despite receiving relatively few votes. This indicates that some high-quality productions remain under-exposed to large audiences.

Task 5: Word Frequency in Titles

This task functioned as a standard MapReduce-style word count executed via Spark DataFrames. We used $regexp_replace$ to strip all non-alphanumeric characters (punctuation) and cast the text to lowercase. The `split` function converted each title into an array of words, and Spark ML's built-in $StopWordsRemover$ was applied to automatically remove common stop words. We then used `explode` to map each title into individual word rows, followed by `groupBy().count()` to act as the reducer.

Insight: After filtering out stop words, our analysis revealed that highly emotional and universal concepts dominate movie titles, with 'love', 'live' and 'story' ranking among the highest frequencies.

Task 6: Genre Diversity in Ratings

To measure rating variability, we `explode`d the genre column to ensure titles belonging to multiple genres were accurately represented. We `clean`ed and `filter`ed out invalid or empty genre values, then `grouped by` genre and utilized the stddev aggregate function as our primary metric for variability. Finally, we sorted the results and identified the genres with the highest and lowest variability.

Insight: We discovered that the 'Horror' genre has the highest rating variability (StdDev: ~1.37), indicating that horror movies are highly polarizing - they are either loved or hated. Conversely, 'Film-Noir' has the lowest variability (StdDev: ~0.49), showing highly consistent user reception.

Task 7: Certification Rating Distribution

We `filter`ed the base dataset to exclude TV shows and removed rows with missing or empty certificate values. We then `group`ed the movies by their certificate (e.g., PG-13, R, TV-MA). By applying multiple aggregate functions (`count` and `avg`) within a single `.agg()` call, we simultaneously retrieved the number of movies and the mean rating per certificate to analyse how certification categories relate to user ratings.

Insight: Most movies fall under TV-MA, TV-14, and R certifications, indicating a strong focus on mature audiences. However, certifications with fewer titles sometimes show higher average ratings, which may reflect niche or higher-quality productions.

Task 8: Comparing TV Shows and Movies

Using the is_tv_show Boolean flag engineered during preprocessing, we `group`ed the dataset into two categories (TV shows and movies). We aggregated the data to calculate the total number of titles, overall average rating, average votes, and total votes per category. This enables a comparison of user engagement ($votes$) and critical reception ($ratings$) between cinematic and television formats. Additionally, we `group`ed the data by ($start_year$, $category$) using the is_tv_show flag to analyze trends in popularity and ratings over time.

Insight: TV shows consistently maintain a significantly higher average rating (~7.35) than movies (~6.40). However, movies dominate in overall user engagement, drawing nearly double the total volume of votes (111M vs. 60M).

Task 9: Certification Analysis

Building upon the distribution logic from Task 7, we analysed movie certifications by `group`ing movies by certificate, counting the number of movies in each category, and calculating the average rating per certification. We then ordered the results in descending order of average rating and selected the certification with the highest average rating.

Insight: Certifications with very few movies sometimes appear to have the highest average rating

Task 10: Comparing Categories Over Time

To analyse chronological trends, we reused the cleaned $start_year$ column extracted during preprocessing and the is_tv_show flag created in Task 1. We `group`ed the dataset by is_tv_show to compare overall average ratings and total votes between movies and TV shows. We then executed a multi-column grouping operation on ($start_year$, is_tv_show) to track how popularity ($total\ votes$) and $ratings$ evolved over time for both categories.

Insight: The number of releases increased significantly after 2010, reflecting the growth of streaming platforms. TV shows consistently maintain higher average ratings, suggesting stronger long-term audience engagement compared to movies.

Core Spark Concepts Utilized

- **Lazy Evaluation & Directed Acyclic Graph (DAG):** Spark does not execute transformations immediately. Instead, it builds a DAG of logical execution steps. Computations were only triggered when actions like `.show()` or `.count()` were invoked, allowing Spark's Catalyst Optimizer to determine the most efficient execution plan.

- **Caching** (`.cache()`)**:** The heavily transformed df_clean DataFrame was cached into memory in Task 1. Because subsequent tasks (2–10) reused this cleaned dataset, caching prevented Spark from re-running expensive regex and string cleaning operations multiple times.

- **Minimizing Shuffles:** Whenever possible, `.filter()` operations were applied before wide transformations such as `.groupBy()` or `.join()`. Reducing the dataset size early minimizes network shuffling and improves performance.

- **Window Functions**: In Task 2, we used a Spark Window function partitioned by genre to rank movies within each group efficiently without requiring manual grouping or multiple passes over the data.